Expense Tracker

A Full-Stack Personal Finance Management Web Application

Student Name	_____	**Technology Stack**	React, Flask, SQLite
Roll No.	_____	**Project Type**	Full-Stack Web Application
Guide Name	_____	**Academic Year**	_____

1. Introduction

Expense Tracker is a full-stack web application that allows users to record, manage, and analyze their personal income and expenses. It provides secure user authentication, complete transaction management, category-wise and date-wise filtering, and a visual dashboard summarizing financial activity through charts and reports.

1.1 Objective

- To design and develop a secure, user-friendly platform for tracking personal income and expenses.
- To provide real-time financial summaries (income, expenses, balance) through a dashboard.
- To implement a RESTful API architecture separating frontend and backend concerns.
- To apply industry-standard practices: password hashing, token-based authentication, input validation, and per-user data isolation.

1.2 Scope

The system supports individual users maintaining their own financial records. Each user can register, log in, add/edit/delete transactions, filter and search records, view category-wise spending, monthly trends, and manage their profile including a profile picture.

2. Existing System vs. Proposed System

Existing System	Proposed System
Manual tracking via notebooks or spreadsheets	Automated, structured digital record-keeping
No real-time financial summary	Live dashboard with income, expense, balance, and charts
Prone to human calculation errors	Server-side validation and automatic calculations
No data security / single-user files	Authenticated, per-user isolated data with hashed passwords
Difficult to search/filter past records	Search and filter by category, type, and date range

3. Technology Stack

Layer	Technology	Purpose
Frontend	React (Vite), React Router	User interface, client-side routing
Backend	Python, Flask	REST API, business logic, authentication
Database	SQLite 3	Persistent storage for users and transactions
Charts	Recharts	Visual reports (bar chart, pie chart)
Testing	pytest	Automated backend test suite (21 tests)
Version Control	Git & GitHub	Source control and collaboration

4. System Architecture

The application follows a three-tier architecture, separating presentation, business logic, and data layers:

```
React Frontend  →  HTTP / JSON (REST API)  →  Flask Backend (Routes → Services → Models)  →  SQLite Database
```

Routes handle HTTP requests, **Services** contain business logic and validation, and **Models** perform raw database queries — keeping each layer's responsibility separate and the codebase maintainable.

5. Modules / Features

- **Authentication** — Register, login, logout, token-based session, secure password hashing.
- **Transaction Management** — Create, read, update, delete income/expense records with validation.
- **Search & Filter** — Filter by category, type, date range; keyword search.
- **Dashboard** — Total income, expenses, balance, transaction count, category breakdown.
- **Reports** — Monthly income vs. expense bar chart and category-wise pie chart.
- **Profile Management** — Update name, change password, upload profile picture.
- **Security** — Per-user data isolation, parameterized SQL queries, environment-based secrets.

6. Database Design

users	transactions
id (PK)	id (PK)
name	user_id (FK → users.id)
email (unique)	amount
password_hash	type (income / expense)
token	category
avatar	description, date
created_at	created_at

Relationship: One user can have many transactions (one-to-many via **user_id** foreign key).

7. Advantages

- Real-time visibility into personal financial health.
- Secure, isolated multi-user support on a single deployment.
- Clean separation of concerns (routes/services/models) for easy maintenance.
- Fully tested backend with an automated test suite.
- Responsive design — usable on both desktop and mobile devices.

8. Future Scope

- Budget setting and alerts when nearing budget limits.
- Recurring transaction automation.
- CSV/PDF export of transaction history.
- Migration to PostgreSQL and cloud deployment (Vercel + Render).
- MCP server integration for AI-assisted expense queries.

9. Conclusion

The Expense Tracker project successfully demonstrates the design and implementation of a secure, full-stack web application using React, Flask, and SQLite. It applies REST API principles, layered backend architecture, authentication, and responsive UI design to deliver a practical personal finance management tool, while reinforcing core full-stack development concepts.